



UniSENAI
Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas

Daniel Tavares dos Anjos

Trabalho Prático 01: Qualidade & Teste de Software

Aplicação Prática da ISO/IEC 25010 com Teste Unitário em Case Real

Daniel Tavares dos Anjos

Trabalho Prático 01: Qualidade & Teste de Software

Aplicação Prática da ISO/IEC 25010 com Teste Unitário em Case Real

Trabalho individual apresentado à disciplina de Qualidade e Teste de Software do Curso Superior de Tecnologia em Análise e Desenvolvimento de Sistemas da UniSENAI.

Professor: Hugo Menezes Barra.

Sumário

1	Introdução	1
1.1	Objetivos	1
2	Fundamentação pela ISO/IEC 25010	1
2.1	Adequação funcional	1
2.2	Confiabilidade	1
2.3	Manutenibilidade	2
3	Implementação do componente	2
4	Estratégia e cenários de teste	2
4.1	Mapeamento dos cenários	3
5	Resultados	3
6	Considerações finais	4

1 Introdução

A qualidade de software não depende apenas da execução sem falhas em um cenário comum. É necessário verificar se as funções entregam os resultados especificados, se dados incorretos são tratados de forma previsível e se mudanças futuras podem ser realizadas com baixo risco de regressão. A ISO/IEC 25010:2023 fornece um modelo de qualidade de produto aplicável à especificação e à avaliação de produtos de software ao longo do ciclo de vida [1].

O estudo de caso deste trabalho é o checkout de um carrinho de compras. O componente soma os itens, aplica descontos progressivos e calcula o frete. Como as decisões mudam exatamente em determinados valores monetários, pequenas falhas de comparação podem produzir cobranças incorretas. Por isso, a análise de valor limite é central para a estratégia de testes.

1.1 Objetivos

O objetivo geral é implementar e validar as regras de negócio da classe `CarrinhoDeCompras` por meio de testes unitários automatizados.

2 Fundamentação pela ISO/IEC 25010

A edição vigente da ISO/IEC 25010 define um modelo de qualidade de produto com nove características e suas subcaracterísticas. O modelo pode apoiar a definição de requisitos, objetivos de teste, critérios de controle de qualidade e critérios de aceitação [1]. Neste estudo de caso, a análise concentra-se nos três pilares solicitados no enunciado.

2.1 Adequação funcional

A adequação funcional representa o grau em que as funções do produto atendem às necessidades declaradas em condições especificadas. No carrinho, isso significa obter valores corretos para soma dos produtos, desconto, frete e total final. A suíte verifica um fluxo sem desconto, todas as transições de desconto e a transição do frete. Dessa forma, não se testa apenas a presença dos métodos, mas a exatidão dos resultados produzidos.

Os pares R\$ 99,99/R\$ 100,00 e R\$ 499,99/R\$ 500,00 demonstram se os operadores de comparação implementam corretamente a expressão “atinge ou ultrapassa”. De modo semelhante, R\$ 199,99/R\$ 200,00 distingue a cobrança de R\$ 20,00 da isenção. Esses testes reduzem o risco de defeitos de fronteira, como usar `>` no lugar de `>=`.

2.2 Confiabilidade

Confiabilidade envolve a capacidade de manter o comportamento esperado sob condições especificadas. A classe rejeita preços iguais ou inferiores a zero e impede o cálculo final quando não existem itens. Em vez de permitir um estado inválido ou devolver um resultado ambíguo, o componente lança `ValueError` com uma mensagem descritiva.

Os testes de exceção confirmam o tipo e o conteúdo essencial da mensagem. Isso torna as falhas previsíveis para as camadas que consomem a classe, que podem capturar a exceção e informar o usuário sem comprometer o restante do checkout.

2.3 Manutenibilidade

A suíte automatizada funciona como uma especificação executável das regras. Cada cenário possui um nome que indica a condição e o resultado esperado; os testes são independentes e utilizam uma pequena função auxiliar para evitar repetição na preparação do carrinho. Seguindo o padrão utilizado nas aulas, os cenários são funções independentes, empregam instruções `assert` com mensagens explicativas e são chamados pelo bloco principal do arquivo. A execução não exige bibliotecas externas, o que simplifica a configuração do ambiente.

Em uma refatoração, os 11 testes podem ser executados em segundos. Se a estrutura interna mudar sem alterar as regras, a suíte continua aprovada; caso uma fronteira, exceção ou fórmula seja quebrada, o teste correspondente identifica imediatamente a regressão. Essa resposta rápida reduz o esforço de localização de defeitos e favorece mudanças controladas.

3 Implementação do componente

A classe `CarrinhoDeCompras` mantém os itens em uma lista de dicionários com nome e preço. A responsabilidade foi dividida em métodos pequenos: adicionar item, somar produtos, calcular desconto, calcular frete e calcular o total final. A fórmula do checkout é:

$$\text{total final} = \text{total dos produtos} - \text{desconto} + \text{frete.} \quad (1)$$

A Tabela 1 resume as decisões implementadas.

Tabela 1: Regras de negócio do checkout

Regra	Condição	Resultado esperado
Preço do item	$\text{preço} \leq 0$	lançamento de <code>ValueError</code>
Desconto inicial	$\text{total} < \text{R\$ } 100,00$	sem desconto
Desconto de 10%	$\text{R\$ } 100,00 \leq \text{total} < \text{R\$ } 500,00$	10% do total dos produtos
Desconto de 20%	$\text{total} \geq \text{R\$ } 500,00$	20% do total dos produtos
Frete padrão	$0 < \text{total} < \text{R\$ } 200,00$	R\$ 20,00
Frete grátis	$\text{total} \geq \text{R\$ } 200,00$	R\$ 0,00
Checkout vazio	nenhum item	lançamento de <code>ValueError</code>

O cálculo de frete devolve zero quando o total dos produtos é zero, conforme o código-base, mas o método público de checkout bloqueia o carrinho vazio antes de produzir o total final. Assim, a regra de exceção permanece garantida no fluxo de checkout.

4 Estratégia e cenários de teste

Os testes seguem a estrutura demonstrada em aula: cada cenário é uma função iniciada por `test_`, prepara seus dados, calcula o resultado e o verifica com `assert`. Cada caso cria seu próprio carrinho, evitando compartilhamento de estado. Para valores resultantes de multiplicações em ponto flutuante, a diferença absoluta é comparada a uma pequena tolerância; para valores exatos de frete e ausência de desconto, é usada igualdade direta. As exceções são verificadas com blocos `try/except`, confirmando tanto o tipo `ValueError` quanto o conteúdo essencial da mensagem.

4.1 Mapeamento dos cenários

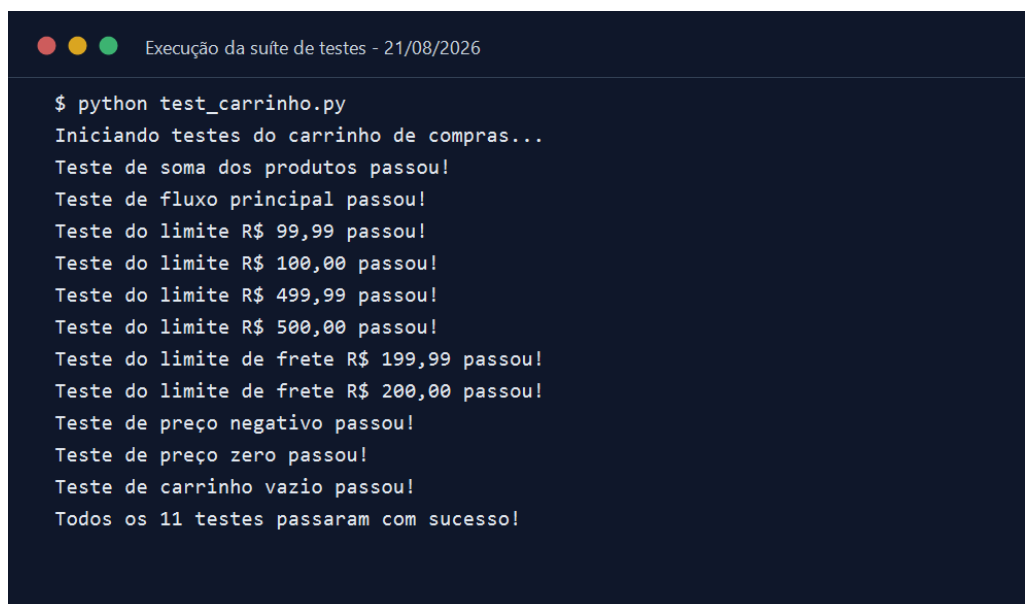
Tabela 2: Mapeamento dos testes unitários

Grupo	Entrada	Comportamento verificado	Resultado esperado
Fluxo principal	R\$ 30,00 + R\$ 45,50	soma de múltiplos itens	R\$ 75,50
Fluxo principal	R\$ 50,00	sem desconto, frete padrão e total final	desconto R\$ 0,00; frete R\$ 20,00; final R\$ 70,00
Limite de desconto	R\$ 99,99	imediatamente abaixo da primeira faixa	desconto R\$ 0,00
Limite de desconto	R\$ 100,00	entrada na faixa de 10%	desconto R\$ 10,00
Limite de desconto	R\$ 499,99	imediatamente abaixo da faixa de 20%	desconto de 10%
Limite de desconto	R\$ 500,00	entrada na faixa de 20%	desconto R\$ 100,00
Limite de frete	R\$ 199,99	imediatamente abaixo da isenção	frete R\$ 20,00
Limite de frete	R\$ 200,00	entrada na faixa de isenção	frete R\$ 0,00
Exceção	preço -R\$ 1,00	rejeição de preço negativo	ValueError
Exceção	preço R\$ 0,00	rejeição de preço nulo	ValueError
Exceção	carrinho vazio	bloqueio do checkout	ValueError

Os valores de fronteira são avaliados em pares, um imediatamente abaixo e outro exatamente no ponto de transição. Essa organização demonstra qual ramo da regra deve ser escolhido em cada lado da fronteira e atende diretamente ao critério de análise de valor limite.

5 Resultados

A suíte foi executada com o comando `python test_carrinho.py`. Os 11 casos foram executados e aprovados. A Figura 1 registra a saída completa do terminal.



```
Execução da suíte de testes - 21/08/2026

$ python test_carrinho.py
Iniciando testes do carrinho de compras...
Teste de soma dos produtos passou!
Teste de fluxo principal passou!
Teste do limite R$ 99,99 passou!
Teste do limite R$ 100,00 passou!
Teste do limite R$ 499,99 passou!
Teste do limite R$ 500,00 passou!
Teste do limite de frete R$ 199,99 passou!
Teste do limite de frete R$ 200,00 passou!
Teste de preço negativo passou!
Teste de preço zero passou!
Teste de carrinho vazio passou!
Todos os 11 testes passaram com sucesso!
```

Figura 1: Execução e aprovação dos 11 testes unitários

O resultado OK confirma que a implementação satisfaz todos os comportamentos codificados na suíte. Em particular, as seis entradas associadas às fronteiras produziram o ramo esperado, e as três entradas inválidas lançaram `ValueError`. Isso fornece evidência objetiva para adequação funcional e confiabilidade. A possibilidade de repetir o mesmo comando após alterações oferece a base prática de manutenibilidade.

O código-fonte, a suíte e as instruções de execução estão disponíveis no repositório público: <https://github.com/danieltanjos/trab1-qa>.

6 Considerações finais

O estudo demonstrou a aplicação concreta de conceitos da ISO/IEC 25010 a um componente pequeno, porém sensível do ponto de vista financeiro. A adequação funcional foi trabalhada por meio de resultados esperados explícitos para os fluxos comuns e para cada fronteira. A confiabilidade foi reforçada pela rejeição previsível de estados inválidos. A manutenibilidade foi apoiada por testes independentes, nomes descritivos, ausência de dependências externas e documentação de execução.

A análise de valor limite mostrou-se especialmente relevante: testar apenas valores distantes das transições não revelaria erros nos operadores de comparação. A combinação de testes normais, limites e exceções forma uma suíte curta, reproduzível e alinhada às regras de negócio. Como evolução futura, o componente poderia adotar um tipo decimal para representar moeda e ampliar os testes para múltiplos itens cuja soma coincida com cada fronteira.

Referências

- [1] INTERNATIONAL ORGANIZATION FOR STANDARDIZATION. **ISO/IEC 25010:2023: Systems and software engineering – Systems and software Quality Requirements and Evaluation (SQuaRE) – Product quality model**. 2. ed. Geneva: ISO, 2023. Disponível em: <https://www.iso.org/standard/78176.html>. Acesso em: 21 ago. 2026.